

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP NHÓM

ĐỀ TÀI

**ỨNG DỤNG LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VÀ ĐA LUỒNG MÔ
PHỎNG BÀI TOÁN PHÁ SẢN (GAMBLER'S RUIN) BẰNG PHƯƠNG
PHÁP MONTE CARLO**

(PHÂN TÍCH TOÁN HỌC TRONG TRÒ CHƠI “BÀU CUA”)

Môn học	: Lập trình hướng đối tượng
Mã lớp	: IT002.Q219
Giảng viên hướng dẫn	: Thầy Vũ Tuấn Hải
Nhóm thực hiện	: Nhóm 6

THÀNH VIÊN NHÓM

STT	Họ và tên	Mã sinh viên	Phân công	Tiến độ
1	Lê Thanh Sang	25521575	Dev chính (Thiết kế kiến trúc và code các luồng chính từ C++ sang Python)	Hoàn thành
2	Trần Ngô Nhật Minh	25521137	Dev phụ (viết các hàm logic nghiệp vụ (Business Logic) theo khung của dev chính)	Hoàn thành
3	Lê Hữu Phúc	25521423	QA Tester & IT Support (chạy chương trình, bắt lỗi và hỗ trợ môi trường)	Hoàn thành
4	Nguyễn Hoàng Bảo Châu	25520188	Slide Maker (thiết kế bài trình chiếu)	Hoàn thành
5	Lê Thị Thu Hiền	25520524	Documenter (viết báo cáo)	Hoàn thành

MỤC LỤC

I. MỞ ĐẦU	7
1. Lý do chọn đề tài	7
2. Mục tiêu đồ án	7
2.1. Mục tiêu lập trình (kỹ thuật C++)	7
2.2. Mục tiêu mô phỏng (toán học thống kê)	8
3. Phạm vi và công cụ thực hiện	9
3.1. Phạm vi nghiên cứu	9
3.2. Công cụ và công nghệ sử dụng	9
II. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH TOÁN HỌC	10
1. Bài toán phá sản (Gambler's Ruin) và phương pháp Monte Carlo	10
1.1. Bài toán phá sản (Gambler's Ruin)	10
1.2. Phương pháp Monte Carlo	11
2. Trò chơi Bầu cua và không gian xác suất	12
2.1. Luật chơi và cơ chế trả thưởng (payout)	12
2.2. Tính toán Lợi thế nhà cái (House Edge) và kỳ vọng (EV)	12
3. Các chiến thuật đặt cược được đưa vào mô phỏng	13
3.1. Chiến thuật Chung thủy (Martingale – Gấp thếp 1 cửa)	14
3.2. Chiến thuật Rải thảm (Spread Betting – Phân bố rủi ro)	14
III. Thiết kế kiến trúc và lập trình hệ thống	15
1. Kiến trúc tổng thể (System Architecture)	15
2. Áp dụng Lập trình Hướng đối tượng (OOP Design Patterns)	19
2.1. Strategy Pattern – Linh hoạt hóa logic đặt cược (BettingStrategy.h)	19
2.2. Factory Pattern – Quản lý khởi tạo người chơi (PlayerFactory.h)	21

3. Quản lý bộ nhớ hiện đại (Modern Memory Management) - Ứng dụng RAII và Smart Pointers (std::unique_ptr).....	22
4. Kỹ thuật Lập trình Đa luồng (Multi-threading)	24
4.1. Khởi tạo luồng động (std::thread::hardware_concurrency).....	24
4.2. Đảm bảo tính ngẫu nhiên (Entropy) bằng static thread_local	24
5. Tối ưu hóa hiệu năng I/O (I/O Performance Optimization).....	25
5.1. Giải quyết nút thắt cổ chai ổ cứng (Disk I/O Bottleneck)	25
5.2. Kỹ thuật Ghi đệm RAM (RAM Buffering) bằng std::ostringstream.....	25
6. Đường ống Dữ liệu Tự động (Automated Data Pipeline) – Tích hợp Python sinh biểu đồ tự động bằng C++ (std::system)	26
IV. Triển khai mô phỏng và kết quả phân tích	27
1. Thiết lập các thông số kịch bản mô phỏng.....	27
1.1. Số vòng mô phỏng.....	27
1.2. Thiết lập đa luồng	27
1.3. Thiết lập vốn ban đầu	28
1.4. Thiết lập chiến lược cược	28
1.5. Điều kiện dừng mô phỏng	28
2. Kịch bản 1: Nhà cái vốn nhỏ (Mô phỏng Variance ngắn hạn).....	29
2.1. Phân tích kết quả "Người chơi phá sản sớm nhất"	29
2.2. Phân tích kết quả "Người chơi trụ lại lâu nhất"	30
2.3. Sự sụp đổ của nhà cái dưới áp lực của chiến thuật Martingale	30
3. Kịch bản 2: Nhà cái vốn lớn (Quy luật số lớn)	31
3.1. Phân tích sự sụt giảm vốn tất yếu của người chơi	31
3.2. Biến động số dư trung bình thực nghiệm	32
4. Đánh giá chung về mức độ rủi ro của các chiến thuật	33
5. Đề xuất chiến lược tối ưu	34

5.1. Góc nhìn Người chơi: Tham gia như thế nào là tối ưu nhất?	34
5.2. Góc nhìn Chủ (Nhà cái): Quản lý trò chơi như thế nào để tối ưu lợi nhuận?.....	34
V. Kết luận và hướng phát triển.....	35
1. Kết quả đạt được về mặt Kỹ thuật Phần mềm.....	35
2. Kết quả đạt được về mặt Thống kê (Bản chất của sòng bạc).....	36
3. Hạn chế còn tồn tại của hệ thống	36
4. Hướng phát triển trong tương lai.....	37
VI. Tổng kết.....	39
1. Tổng kết.....	39
2. Tuyên bố miễn trừ trách nhiệm	39
3. Hướng dẫn cài đặt và sử dụng mã nguồn.....	39
VII. Tài liệu tham khảo.....	41

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Từ viết tắt	Nghĩa đầy đủ (Tiếng Anh)	Diễn giải (Tiếng Việt)
OOP	Object-Oriented Programming	Lập trình hướng đối tượng
EV	Expected Value	Giá trị kỳ vọng
HE	House Edge	Lợi thế nhà cái
RNG	Random Number Generator	Bộ sinh số ngẫu nhiên
RAII	Resource Acquisition Is Initialization	Nguyên lý quản lý tài nguyên gắn với vòng đời đối tượng

I. MỞ ĐẦU

1. Lý do chọn đề tài

Trong thực tế, các trò chơi mang tính may rủi luôn thu hút sự quan tâm của nhiều người nhờ tính đơn giản và khả năng mang lại lợi nhuận nhanh chóng trong ngắn hạn. Một trong những trò chơi phổ biến tại Việt Nam là Bầu Cua, với cơ chế dựa hoàn toàn vào yếu tố ngẫu nhiên. Tuy nhiên, phần lớn người chơi thường đánh giá kết quả dựa trên cảm tính mà chưa nhận thức đầy đủ về bản chất xác suất và các quy luật toán học chi phối trò chơi.

Trong lĩnh vực xác suất – thống kê, mô hình Gambler’s Ruin được sử dụng để mô tả quá trình suy giảm tài sản của người chơi khi tham gia các trò chơi có lợi thế nghiêng về phía đối phương. Mô hình này cho thấy rằng, mặc dù có thể tồn tại những khoảng thời gian thắng lợi, nhưng về lâu dài, khả năng người chơi bị thua lỗ hoặc phá sản là rất cao. Việc áp dụng mô hình này vào trò chơi Bầu Cua giúp làm rõ hơn bản chất “càng chơi càng thua” dưới góc nhìn khoa học.

Ngoài ra, với sự phát triển của công nghệ, các phương pháp mô phỏng như Monte Carlo cho phép tái hiện hàng triệu đến hàng trăm triệu lần thử ngẫu nhiên trong thời gian ngắn, từ đó cung cấp dữ liệu đủ lớn để phân tích xu hướng xác suất một cách chính xác. Đồng thời, việc kết hợp lập trình hướng đối tượng (OOP) và kỹ thuật đa luồng không chỉ giúp xây dựng hệ thống mô phỏng có cấu trúc rõ ràng mà còn tối ưu hiệu năng xử lý, đáp ứng yêu cầu tính toán quy mô lớn.

Xuất phát từ những lý do trên, đề tài “Ứng dụng lập trình hướng đối tượng và đa luồng mô phỏng bài toán phá sản (Gambler’s Ruin) bằng phương pháp Monte Carlo (phân tích toán học trong trò chơi Bầu Cua)” được lựa chọn nhằm mục đích nghiên cứu, mô phỏng và kiểm chứng một cách định lượng xu hướng thắng thua trong trò chơi. Qua đó, đề tài không chỉ góp phần củng cố kiến thức lập trình và tối ưu hệ thống, mà còn cung cấp một góc nhìn khoa học về rủi ro và xác suất trong các trò chơi mang tính may rủi.

2. Mục tiêu đồ án

2.1. Mục tiêu lập trình (kỹ thuật C++)

- Thiết kế và xây dựng hệ thống mô phỏng theo hướng lập trình hướng đối tượng (OOP) với các thành phần rõ ràng như người chơi, nhà cái, chiến lược cược và bộ sinh số ngẫu nhiên.

- Xây dựng các lớp đối tượng chính như BettingStrategy, House, Player, PlayerFactory, Simulator, Strategies
- Ứng dụng kỹ thuật đa luồng (Multithreading) để tối ưu hiệu năng, cho phép mô phỏng hàng triệu đến hàng trăm triệu ván chơi trong thời gian ngắn.
- Xây dựng cơ chế sinh số ngẫu nhiên độc lập theo từng luồng nhằm đảm bảo tính chính xác và tránh xung đột tài nguyên.
- Áp dụng các nguyên lý quản lý bộ nhớ hiện đại nhằm đảm bảo hệ thống hoạt động ổn định và không rò rỉ tài nguyên trong quá trình mô phỏng quy mô lớn.
- Tích hợp quy trình xử lý dữ liệu tự động, bao gồm xuất dữ liệu mô phỏng và trực quan hóa kết quả bằng công cụ phân tích.

2.2. Mục tiêu mô phỏng (toán học thống kê)

- Xây dựng mô hình mô phỏng dựa trên phương pháp Monte Carlo để thực hiện số lượng lớn ván chơi với kết quả ngẫu nhiên
- Tính toán và kiểm chứng giá trị kỳ vọng (Expected Value - EV) của trò chơi, qua đó xác định lợi thế toán học của nhà cái.
- Phân tích xác suất xảy ra các kết quả (0, 1, 2, 3 lần trùng) và tác động của chúng đến lợi nhuận của người chơi.
- Đánh giá sự biến động ngắn hạn (variance) và xu hướng dài hạn dựa trên định luật số lớn (Law of Large Numbers).
- Mô phỏng quá trình phá sản của người chơi theo thời gian, từ đó chứng minh tính tất yếu của hiện tượng “càng chơi càng thua”.
- So sánh hiệu quả của các chiến lược cược khác nhau (ví dụ: chung thủy (Martingale), rải thảm (Spread Betting)...) và đánh giá khả năng chống lại lợi thế nhà cái.

Thông qua việc đạt được các mục tiêu trên, đồ án không chỉ giúp củng cố kiến thức về lập trình nâng cao (OOP, đa luồng, tối ưu hiệu năng) mà còn góp phần làm rõ bản chất toán học của các trò chơi may rủi, cung cấp một góc nhìn khoa học và định lượng về rủi ro tài chính trong thực tế.

3. Phạm vi và công cụ thực hiện

3.1. Phạm vi nghiên cứu

Đề tài tập trung xây dựng hệ thống mô phỏng để phân tích rủi ro tài chính và xu hướng thắng thua trong trò chơi Bầu Cua thông qua mô hình Gambler's Ruin và phương pháp Monte Carlo. Phạm vi nghiên cứu được giới hạn như sau:

- Mô phỏng trò chơi Bầu Cua với 3 xúc xắc độc lập, mỗi xúc xắc có 6 mặt, tuân theo xác suất đồng đều.
- Xét các chiến lược cược tiêu biểu: chung thủy (gấp thếp 1 cửa) và rải thảm (chia 3 cửa)
- Phân tích kết quả dựa trên các chỉ số: giá trị kỳ vọng (Expected Value-EV), tỉ lệ thắng/thua, biến động ngắn hạn (variance) xu hướng dài hạn theo luật số lớn
- Mô phỏng qua trình thay đổi vốn của người chơi và xác suất dẫn đến phá sản theo thời gian
- Xây dựng hai kịch bản chính: nhà cái có vốn nhỏ và nhà cái có vốn lớn

Đề tài không xét đến các yếu tố như tâm lý người chơi, gian lận hay các biến thể của luật chơi trong thực tế.

3.2. Công cụ và công nghệ sử dụng

- Ngôn ngữ lập trình C++ (C++ 17)
 - Xây dựng hệ thống mô hình chính với hiệu năng cao.
 - Sử dụng thư viện chuẩn như `std::thread` để triển khai đa luồng
 - Áp dụng `thread_local std::mt19937` để sinh số ngẫu nhiên độc lập cho từng luồng.
 - Quản lý bộ nhớ bằng Smart Pointer (`std::unique_ptr`) theo nguyên lý RAII cho các đối tượng Player, House và BettingStrategy.
- Kỹ thuật đa luồng (Multithreading)
 - Tận dụng toàn bộ tài nguyên CPU thông qua cơ chế phân chia công việc động.
 - Cho phép mô phỏng hàng triệu đến hàng trăm triệu ván chơi trong thời gian ngắn.
 - Thiết kế theo hướng hạn chế sử dụng mutex nhằm tránh xung đột tài nguyên.
- Lập trình hướng đối tượng (OOP)

- Áp dụng các mẫu thiết kế (Design Patterns)
 - Strategy Pattern cho các chiến lược cược
 - Factory Pattern cho khởi tạo đối tượng
- Tổ chức hệ thống thành các lớp như BettingStrategy, House, Player, PlayerFactory, Simulator, Strategies

- Python và thư viện phân tích dữ liệu

- Sử dụng Python (phiên bản 3.x) kết hợp các thư viện: pandas để xử lý dữ liệu, matplotlib để trực quan hóa, numpy để hỗ trợ tính toán
- Tự động đọc dữ liệu từ file CSV và tạo biểu đồ phân tích.

- Môi trường phát triển

- Visual Studio (khuyến nghị phiên bản 2022) để biên dịch và chạy chương trình C++.

- Định dạng dữ liệu

- Dữ liệu mô phỏng được lưu dưới dạng file CSV để thuận tiện cho việc xử lý và trực quan hóa.

Thông qua việc giới hạn phạm vi rõ ràng và sử dụng các công cụ phù hợp, đề tài đảm bảo tính khả thi, hiệu quả và độ chính xác trong quá trình mô phỏng và phân tích.

II. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH TOÁN HỌC

1. Bài toán phá sản (Gambler's Ruin) và phương pháp Monte Carlo

1.1. Bài toán phá sản (Gambler's Ruin)

Bài toán Gambler's Ruin là một mô hình kinh điển trong xác suất – thống kê, dùng để mô tả quá trình một người chơi tham gia vào các trò chơi có tính ngẫu nhiên và có nguy cơ mất toàn bộ số vốn ban đầu.

Trong mô hình cơ bản, người chơi bắt đầu với một số vốn nhất định và tham gia vào các ván chơi liên tiếp. Ở mỗi ván, người chơi có thể thắng hoặc thua với một xác suất nhất định. Nếu trò chơi là công bằng (xác suất thắng bằng xác suất thua), người chơi có thể duy trì vốn trong một khoảng

thời gian. Tuy nhiên, khi xác suất thắng nhỏ hơn xác suất thua, hoặc khi giá trị kỳ vọng (Expected Value) là âm, người chơi sẽ có xu hướng mất dần vốn theo thời gian.

Trong trường hợp áp dụng vào trò chơi Bầu Cua, do tồn tại lợi thế toán học của nhà cái (House Edge), giá trị kỳ vọng của người chơi là âm. Điều này dẫn đến một kết luận quan trọng: mặc dù có thể thắng trong ngắn hạn do yếu tố ngẫu nhiên, nhưng về dài hạn, người chơi gần như chắc chắn sẽ bị thua lỗ hoặc phá sản.

Bài toán Gambler's Ruin không chỉ giúp mô tả quá trình suy giảm tài chính mà còn là cơ sở để phân tích các yếu tố như thời gian phá sản, xác suất phá sản và ảnh hưởng của chiến lược cược. Đây là nền tảng lý thuyết quan trọng để xây dựng mô hình mô phỏng trong đề tài.

1.2. Phương pháp Monte Carlo

Phương pháp mô phỏng Monte Carlo là một kỹ thuật được sử dụng để mô hình hóa xác suất của các kết quả khác nhau trong những quá trình khó dự đoán do sự tác động của các biến ngẫu nhiên. Đây là một phương pháp giúp phân tích và đánh giá ảnh hưởng của rủi ro cũng như sự không chắc chắn trong các hệ thống. Phương pháp này có thể được áp dụng cho nhiều bài toán trong nhiều lĩnh vực khác nhau như đầu tư tài chính, kinh doanh, vật lý và kỹ thuật. Ngoài ra, phương pháp này còn được gọi là mô phỏng đa xác suất, do dựa trên việc thực hiện nhiều lần thử nghiệm ngẫu nhiên để ước lượng kết quả. [1]

Trong bối cảnh của đề tài, phương pháp Monte Carlo được sử dụng để mô phỏng hàng triệu đến hàng trăm triệu ván chơi Bầu Cua. Mỗi ván chơi được xem như một phép thử độc lập, trong đó kết quả được xác định thông qua bộ sinh số ngẫu nhiên.

Quy trình mô phỏng bằng phương pháp Monte Carlo bao gồm các bước cơ bản sau:

- Xây dựng mô hình toán học của trò chơi.
- Sinh các giá trị ngẫu nhiên để mô phỏng kết quả xúc xắc.
- Lặp lại quá trình mô phỏng trong số lượng lớn ván chơi.
- Thu thập và phân tích dữ liệu đầu ra.

Ưu điểm lớn nhất của phương pháp Monte Carlo là khả năng xử lý các bài toán phức tạp mà khó có thể giải bằng phương pháp lý thuyết thuần túy. Khi số lượng lần mô phỏng đủ lớn, kết quả thu được sẽ tiệm cận với giá trị thực tế theo định luật số lớn (Law of Large Numbers).

2. Trò chơi Bầu cua và không gian xác suất

2.1. Luật chơi và cơ chế trả thưởng (payout)

Bầu Cua là một trò chơi sử dụng 3 xúc xắc, mỗi xúc xắc có 6 mặt tương ứng với 6 biểu tượng: bầu, cua, tôm, cá, gà, nai. Trong mỗi ván chơi, người chơi đặt cược vào một hoặc nhiều biểu tượng trước khi tiến hành lắc xúc xắc.

Sau khi lắc, kết quả được xác định dựa trên số lần xuất hiện của biểu tượng mà người chơi đã chọn. Cơ chế trả thưởng được quy định như sau:

- Nếu không có mặt nào trùng với biểu tượng đã cược → người chơi thua toàn bộ số tiền cược.
- Nếu xuất hiện 1 mặt trùng → người chơi nhận lại tiền cược và thắng thêm số tiền bằng 1 lần cược.
- Nếu xuất hiện 2 mặt trùng → người chơi thắng gấp 2 lần số tiền cược.
- Nếu xuất hiện 3 mặt trùng → người chơi thắng gấp 3 lần số tiền cược.

Có thể thấy rằng, số tiền thắng tỷ lệ thuận với số lần xuất hiện của biểu tượng được chọn. Tuy nhiên, xác suất xuất hiện nhiều lần trùng là khá thấp, điều này ảnh hưởng trực tiếp đến lợi nhuận kỳ vọng của người chơi.

2.2. Tính toán Lợi thế nhà cái (House Edge) và kỳ vọng (EV)

Để phân tích lợi thế toán học trong trò chơi, xét trường hợp người chơi đặt cược 1 đơn vị tiền vào một biểu tượng bất kỳ. Với 3 xúc xắc độc lập, mỗi xúc xắc có 6 mặt, không gian xác suất có tổng cộng 216 khả năng xảy ra.

Các trường hợp có thể xảy ra bao gồm:

- Thua (0 lần trùng):

$$\text{Xác suất} = (5/6)^3 = 125/216 \approx 57.87\%$$

$$\text{Lợi nhuận} = -1$$

- Thắng 1 lần trùng:

$$\text{Xác suất} = 75/216 \approx 34.72\%$$

$$\text{Lợi nhuận} = +1$$

- Thắng 2 lần trùng:

$$\text{Xác suất} = 15/216 \approx 6.94\%$$

$$\text{Lợi nhuận} = +2$$

- Thắng 3 lần trùng:

$$\text{Xác suất} = 1/216 \approx 0.46\%$$

$$\text{Lợi nhuận} = +3$$

Từ đó, kỳ vọng toán học (Expected Value - EV) của một lần cược được tính như sau:

$$EV = (-1 \times 125/216) + (1 \times 75/216) + (2 \times 15/216) + (3 \times 1/216)$$

$$= (-125 + 75 + 30 + 3) / 216$$

$$= -17/216 \approx -0.0787$$

Kết quả cho thấy giá trị kỳ vọng là âm, nghĩa là trung bình mỗi lần cược, người chơi sẽ mất khoảng 0.0787 đơn vị tiền.

Từ đó, ta xác định được lợi thế của nhà cái (House Edge): $\text{House Edge} \approx 7.87\%$

Điều này có nghĩa là trong dài hạn, người chơi sẽ mất khoảng 7.87 đơn vị cho mỗi 100 đơn vị tiền cược. Đây là một mức lợi thế khá cao, cho thấy trò chơi được thiết kế có lợi cho nhà cái.

Do đó, mặc dù người chơi có thể thắng trong ngắn hạn do yếu tố ngẫu nhiên, nhưng về lâu dài, với giá trị kỳ vọng âm, người chơi sẽ có xu hướng thua lỗ. Đây cũng chính là cơ sở để áp dụng mô hình Gambler's Ruin trong việc phân tích và mô phỏng.

3. Các chiến thuật đặt cược được đưa vào mô phỏng

Nhằm đánh giá tác động của phương pháp đặt cược đến kết quả tài chính của người chơi, nhóm đã chọn hai chiến thuật phổ biến là chiến thuật Chiến thuật Chung thủy (Martingale – Gấp thếp 1 cửa)

và chiến thuật Rải thảm (Spread Betting – Phân bố rủi ro) để đưa vào mô phỏng. Hai chiến thuật này đại diện cho hai cách tiếp cận khác nhau: một bên là tập trung vào việc gỡ lỗ nhanh (tăng rủi ro) và một bên là phân tán rủi ro.

3.1. Chiến thuật Chung thủy (Martingale – Gấp thếp 1 cửa)

Chiến lược Martingale là phương pháp dựa trên đặt cược liên quan đến việc tăng gấp đôi mức đặt cược sau mỗi lần thua lỗ nhằm nỗ lực thu hồi mọi khoản lỗ và cuối cùng tạo ra lợi nhuận [2]. Trong chiến thuật này, người chơi chỉ đặt vào một cửa duy nhất và thực hiện theo quy tắc:

- Nếu thắng: quay về mức cược ban đầu.
- Nếu thua: tăng gấp đôi số tiền cược ở ván tiếp theo.

Mục tiêu của chiến thuật này là khi thắng, người chơi sẽ bù lại toàn bộ số tiền đã thua trước đó và thu về một khoản lợi bằng mức cược ban đầu.

Tuy nhiên chiến thuật này tồn tại rủi ro như sau: Khi gặp chuỗi thua liên tiếp, số tiền cược sẽ tăng lên theo cấp số nhân. Người chơi phải chi ra số tiền lớn dẫn đến nhanh chóng hết vốn dẫn đến phá sản. Trong trò chơi có kỳ vọng âm như trò Bầu Cua, chiến thuật này không làm thay đổi giá trị kỳ vọng ($EV < 0$), do đó không thể giúp người chơi thắng khi chơi dài hạn.

Trong mô phỏng, chiến thuật Martingale thể hiện rõ sự biến động lớn (variance cao), có thể mang lại lợi nhuận nhanh trong ngắn hạn nhưng tiềm ẩn nguy cơ phá sản rất cao, phù hợp để phân tích theo mô hình Gambler's Ruin.

3.2. Chiến thuật Rải thảm (Spread Betting – Phân bố rủi ro)

Chiến thuật rải thảm là phương pháp đặt cược phân tán vốn vào nhiều cửa khác nhau nhằm giảm thiểu rủi ro trong từng ván chơi. Trong đề tài này, người chơi chia vốn và đặt cược đồng thời vào 3 cửa khác nhau (được chọn ngẫu nhiên) trong mỗi ván.

Mục tiêu của chiến thuật này là giảm xác suất thua hoàn toàn trong một ván chơi, tăng khả năng có ít nhất một cửa thắng, ổn định tài chính của người chơi, hạn chế các biến động lớn đồng thời kéo dài thời gian tham gia trò chơi.

Tuy nhiên chiến thuật này tồn tại rủi ro như sau: Do vốn chia nhỏ cho nhiều cửa khác nhau nên mức thắng mỗi ván không cao. Khi xảy ra chuỗi thua liên tiếp người chơi khó có thể thu hồi lại số

vốn đã mất. Với một trò chơi có kỳ vọng âm như Bầu Cua, dù phân bố vốn vào nhiều cửa, người chơi vẫn chịu ảnh hưởng của lợi thế nhà cái (House Edge), điều này có nghĩa là về lâu về dài tổng tài sản của người chơi có xu hướng giảm.

Trong mô phỏng, chiến thuật rải thảm cho thấy đường biến động vốn “mượt” hơn so với Martingale, ít xảy ra sự sụt giảm đột ngột. Tuy nhiên, khi số lượng ván chơi tăng lên, kết quả vẫn hội tụ về xu hướng giảm dần, phản ánh đúng bản chất xác suất của trò chơi.

III. Thiết kế kiến trúc và lập trình hệ thống

1. Kiến trúc tổng thể (System Architecture)

Hệ thống được xây dựng nhằm mô phỏng quá trình chơi trò Bầu Cua dưới góc nhìn xác suất và tài chính, kết hợp giữa mô hình Gambler’s Ruin và phương pháp Monte Carlo. Kiến trúc tổng thể được thiết kế theo hướng mô-đun hóa, đảm bảo khả năng mở rộng, hiệu năng cao và dễ dàng phân tích dữ liệu.

Hệ thống gồm các thành phần chính như sau:

- Player: Đại diện cho người chơi tham gia mô phỏng, quản lý số vốn và thực hiện đặt cược theo chiến lược.
- House: Quản lý nguồn vốn của nhà cái và xử lý kết quả thắng/thua của người chơi.
- Betting Strategy: Xác định cách thức đặt cược của người chơi
- Strategies: Cài đặt chiến lược Martingale và Spread betting
- PlayerFactory: Chịu trách nhiệm khởi tạo đối tượng người chơi với chiến lược tương ứng.
- Simulator: Thành phần trung tâm của hệ thống, điều phối toàn bộ quá trình mô phỏng như chạy nhiều ván chơi, kết hợp Player, House và Strategy, thu thập và ghi dữ liệu

Hệ thống hoạt động theo quy trình như sau:

❖ Phân xưởng 1: Trạm Mô phỏng Tốc độ cao (Viết bằng C++)

Đây là nơi diễn ra các ván cược thực tế. Vì cần chơi hàng trăm triệu ván, giai đoạn này dùng C++ và chia làm nhiều bộ phận

- Người Quản đốc (File main.cpp)

- **Nhiệm vụ:** Quản lý chung và giao việc. Quản đốc biết phải thực hiện tổng cộng 100.000.000 ván cược.
- **Cách làm việc:** Quản đốc kiểm tra xem CPU của máy tính có bao nhiêu số luồng xử lý - threads. Sau đó, ông chia đều lượng công việc cho các nhân công: một nửa nhân công chuyên thử kịch bản "Nhà cái mang ít tiền" (vốn 15.000), nửa còn lại thử kịch bản "Nhà cái mang nhiều tiền" (vốn 6.000.000). Cuối cùng, ông ra lệnh cho tất cả làm việc cùng lúc (chạy song song) để tiết kiệm thời gian.

- Bàn Chơi Game (đối tượng) (File Simulator.cpp & Simulator.h)

- **Nhiệm vụ:** Nơi xúc xắc được tung và tiền được trao đổi.
- **Cách làm việc:** Trên bàn chơi có sẵn một Nhà Cái và các Người Chơi. Mỗi ván, bàn chơi sẽ tung 3 viên xúc xắc ngẫu nhiên (chọn từ Bầu, Cua, Tôm, Cá, Gà, Nai). Nó tự động thu tiền người thua và trả tiền người thắng. Trò chơi trên bàn này chỉ dừng lại khi Nhà Cái phá sản hoặc tất cả người chơi đều hết sạch tiền.

- Các Thực Thể Tham Gia (đối tượng) (File House.h & Player.h)

- **Nhà Cái (House):** Chỉ quan tâm đến việc giữ tiền, thu tiền và đền tiền. Nó sẽ báo động nếu số dư rơi xuống mức 0 (phá sản).
- **Người Chơi (Player):** Mang theo 800.000 tiền vốn vào bàn và được gán một "bộ não" chiến thuật riêng biệt.

- Bộ Não Chiến Thuật (File BettingStrategy.h & Strategies.h): Dự án tạo ra 2 kiểu người chơi với 2 phong cách đánh bạc kinh điển

- **Người chơi "Chung Thủy":** Chỉ nhắm mắt đánh đúng 1 cửa ngẫu nhiên. Nếu thua, ván sau họ sẽ nhân đôi số tiền cược lên (gấp thếp) để gỡ gạc, tiếp tục cho đến khi hết vốn.
- **Người chơi "Rải Thảm":** An toàn hơn, họ lấy số tiền cược chia đều làm 3 phần và đặt rải rác vào 3 cửa ngẫu nhiên khác nhau trên bàn.

❖ **Phân xưởng 2: Kho Lưu Trữ Hồ Sơ (Các file CSV)**

- Khi bàn chơi game hoạt động, cần có người ghi chép lại biến động tài sản để sau này báo cáo.

- **Nhiệm vụ:** Thay vì in ra màn hình sẽ làm chậm tiến độ, hệ thống ghi chép âm thầm vào các file văn bản (file .csv).

- **Cách ghi chép:** Cứ sau mỗi ván, thư ký ghi lại: Số ván, Tiền của Chung Thủy, Tiền của Rải Thảm, Tiền của Nhà Cái. Để ghi cực nhanh, thư ký không viết vào sổ ngay mà ghi tạm ra nháp, đủ 10.000 ván mới chép vào sổ một lần. Các cuốn sổ được chia theo kịch bản: sổ nhà cái vốn "nhỏ" và sổ nhà cái vốn "lớn".

❖ **Phân xưởng 3: Phòng Tổng Hợp & Đồ Họa (Viết bằng Python)**

Khi toàn bộ nhân công C++ báo cáo đã chơi xong hàng trăm triệu ván cược, Quản đốc C++ sẽ tự động đánh gọi phòng Python làm việc.

- Bộ phận Thu thập (Data Aggregator)

- Python đi vòng quanh kho lưu trữ, gom tất cả các cuốn sổ (file csv) lẻ tẻ của kịch bản 1 (Nhà cái nhỏ) và kịch bản 2 (Nhà cái lớn) lại với nhau.

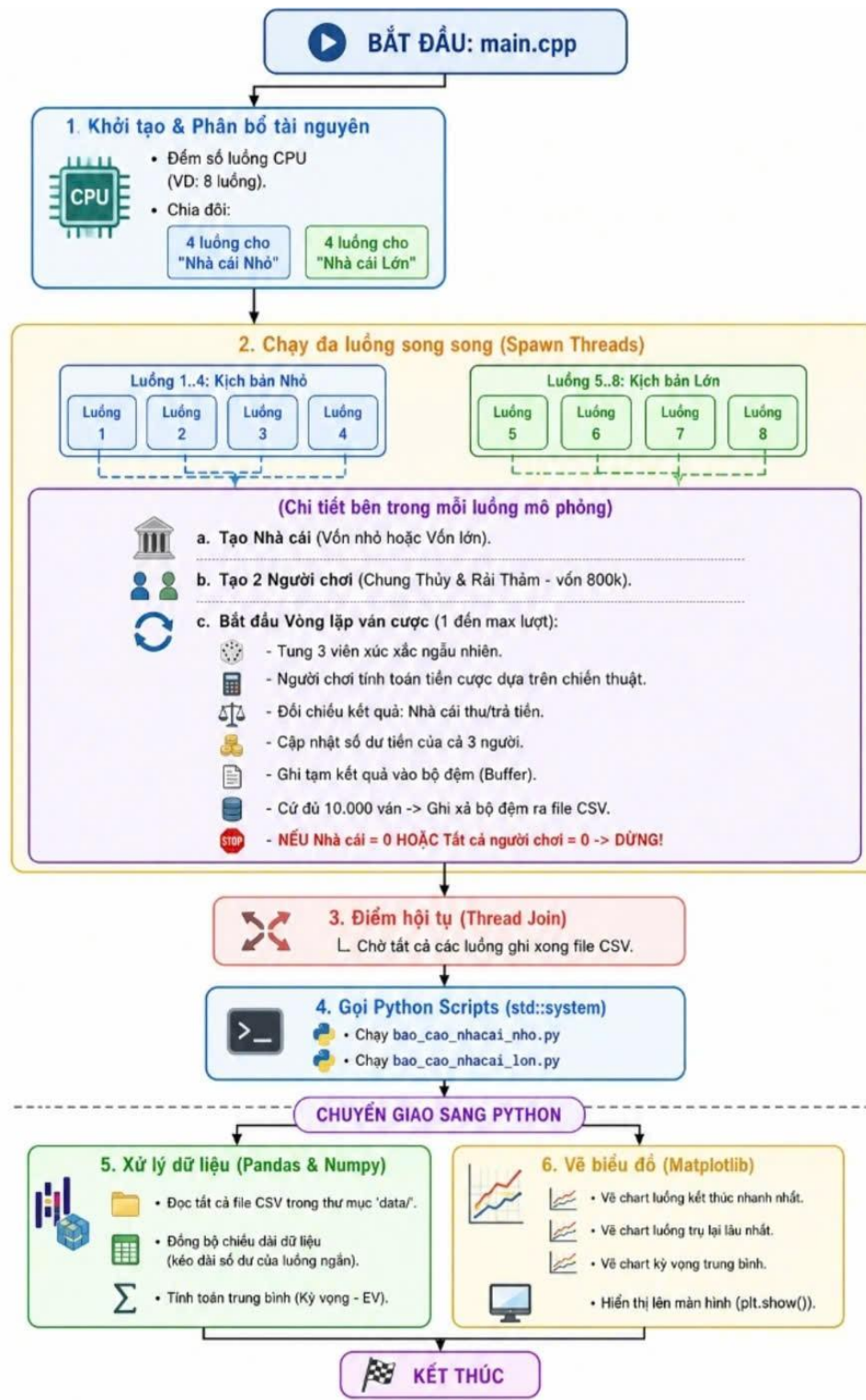
- Bộ phận Chính lý Dữ liệu (Xử lý Mảng)

- Trong thực tế có bàn chơi thì người chơi cháy túi quá nhanh (chỉ vài trăm ván), nhưng có bàn họ lại may mắn sống rất lâu (hàng chục ngàn ván). Để so sánh công bằng toàn bộ các bàn chơi, Python dùng một mẹo: Những ai cháy túi sớm, nó sẽ kéo dài số dư tiền (thường là 0 đồng hoặc số tiền rất ít) cho đến tận mức ván cược dài nhất của người sống sót lâu nhất. Nhờ vậy, máy tính mới có thể tính được trung bình biến động tài sản.

- Bộ phận In Ấn Biểu Đồ (File bao_cao_nhacai_*.py): Cuối cùng, Python vẽ ra 3 bức tranh trực quan cho mỗi kịch bản để người xem hiểu ngay vấn đề mà không cần đọc số liệu:

- **Bức tranh Bi kịch (Nhanh nhất):** Vẽ lại bàn chơi mà người chơi xui xẻo mất trắng tiền trong thời gian ngắn nhất.
- **Bức tranh Phép màu (Lâu nhất):** Vẽ lại bàn chơi mà người chơi cực kỳ may mắn, giằng co với nhà cái lâu nhất.
- **Bức tranh Sự thật Toán học (Kỳ vọng trung bình):** Đây là bức tranh quan trọng nhất. Nó cộng gộp kết quả của hàng trăm triệu ván lại để vẽ ra một đường cong xu hướng. Đường cong này thường cho thấy dù bạn dùng chiến thuật gì, về đường dài, tiền của bạn cũng sẽ từ từ đi xuống 0 (bị toán học bào mòn), trong khi tiền của Nhà Cái đi lên.

Sơ đồ mô tả



2. Áp dụng Lập trình Hướng đối tượng (OOP Design Patterns)

2.1. Strategy Pattern – Linh hoạt hóa logic đặt cược (BettingStrategy.h)

Mục tiêu của phần này là đóng gói các thuật toán đặt cược khác nhau thành các lớp riêng biệt. Trong bài toán, mỗi chiến lược cược (Martingale, Spread Betting) có logic riêng nhưng đều tuân theo cùng một giao diện chung BettingStrategy.

Cách triển khai hệ thống:

- File BettingStrategy.h định nghĩa lớp trừu tượng (abstract class) cho các chiến lược cược.
- File Strategies.h cài đặt cụ thể các chiến lược như Martingale (gấp thếp) và Spread Betting (rải thảm)

Người chơi (Player) sẽ sử dụng một đối tượng Strategy để quyết định cách đặt cược trong mỗi ván.

Code minh họa

- BettingStrategy.h

```
1  #pragma once
2  #include <map>
3
4  enum class Symbol { BAU, CUA, TOM, CA, GA, NAI };
5
6  class BettingStrategy {
7  public:
8      virtual ~BettingStrategy() = default;
9      virtual std::map<Symbol, double> calculateBets(double balance, double baseBet, int losses) const = 0;
10 };
```

Trong hệ thống, lớp BettingStrategy được xây dựng dưới dạng lớp trừu tượng, định nghĩa phương thức calculateBets. Phương thức này trả về một cấu trúc dữ liệu ánh xạ giữa các biểu tượng cược và số tiền tương ứng. Các chiến lược cụ thể như Martingale và Spread Betting sẽ kế thừa và cài đặt phương thức này với logic riêng, từ đó cho phép hệ thống linh hoạt thay đổi chiến lược mà không ảnh hưởng đến các thành phần khác

- Strategies.h

```
3  #include <cmath>
4  #include <algorithm>
5  #include <random>
6  #include <vector>
7
8  class ChungThuyStrategy : public BettingStrategy {
9  public:
10     std::map<Symbol, double> calculateBets(double balance, double baseBet, int losses) const override {
11         std::map<Symbol, double> bets;
12         double amount = baseBet * std::pow(2, losses);
13         amount = std::min(amount, balance);
14
15         if (amount > 0) {
16             static thread_local std::mt19937 rng(std::random_device{}());
17             std::uniform_int_distribution<int> dist(0, 5);
18
19             Symbol randomSymbol = static_cast<Symbol>(dist(rng));
20             bets[randomSymbol] = amount;
21         }
22         return bets;
23     }
24 };
25
```

```
26 class RaiThamStrategy : public BettingStrategy {
27 public:
28     std::map<Symbol, double> calculateBets(double balance, double baseBet, int losses) const override {
29         std::map<Symbol, double> bets;
30         double splitBet = std::min(baseBet, balance / 3.0);
31         if (splitBet > 0) {
32             static thread_local std::mt19937 rng(std::random_device{}());
33
34             std::vector<Symbol> allSymbols = {
35                 Symbol::BAU, Symbol::CUA, Symbol::TOM,
36                 Symbol::CA, Symbol::GA, Symbol::NAI
37             };
38
39             std::shuffle(allSymbols.begin(), allSymbols.end(), rng);
40
41             bets[allSymbols[0]] = splitBet;
42             bets[allSymbols[1]] = splitBet;
43             bets[allSymbols[2]] = splitBet;
44         }
45         return bets;
46     }
47 };

```

Trong hệ thống, các chiến lược cược cụ thể được triển khai trong file Strategies.h, bao gồm hai chiến lược chính là Chung Thủy (Martingale) và Rải Thảm (Spread Betting).

Lớp ChungThuyStrategy thực hiện chiến thuật gấp thếp, trong đó số tiền cược được tăng theo cấp số nhân dựa trên số lần thua liên tiếp. Mỗi ván, người chơi chỉ đặt cược vào một cửa duy nhất được chọn ngẫu nhiên.

Ngược lại, lớp RaiThamStrategy thực hiện chiến thuật phân tán rủi ro bằng cách chia số tiền cược thành ba phần bằng nhau và đặt lên ba cửa khác nhau được chọn ngẫu nhiên.

Cả hai chiến lược đều cài đặt phương thức calculateBets, trả về cấu trúc dữ liệu ánh xạ giữa các biểu tượng và số tiền cược tương ứng. Điều này đảm bảo tính linh hoạt và cho phép hệ thống dễ dàng thay đổi chiến lược trong quá trình mô phỏng.

2.2. Factory Pattern – Quản lý khởi tạo người chơi (PlayerFactory.h)

Mục tiêu của phần này là tách biệt quá trình khởi tạo đối tượng người chơi khỏi phần logic chính của chương trình. Điều này giúp hệ thống dễ dàng mở rộng khi cần thêm loại người chơi hoặc chiến lược mới.

Cách triển khai hệ thống:

- File PlayerFactory.h chịu trách nhiệm tạo đối tượng Player
- Player được gán chiến lược cược ngay khi khởi tạo

Code minh họa

- PlayerFactory.h

```
1  #pragma once
2  #include "Player.h"
3  #include "Strategies.h"
4  #include <memory>
5  #include <string>
6
7  class PlayerFactory {
8  public:
9      enum class Type { CHUNG_THUY, RAI_THAM };
10
11     static std::unique_ptr<Player> createPlayer(Type type, const std::string& name, double balance) {
12         double defaultBaseBet = 100.0;
13         std::unique_ptr<BettingStrategy> strat;
14
15         switch (type) {
16             case Type::CHUNG_THUY:
17                 strat = std::make_unique<ChungThuyStrategy>(); break;
18             case Type::RAI_THAM:
19                 strat = std::make_unique<RaiThamStrategy>(); break;
20         }
21         return std::make_unique<Player>(name, balance, defaultBaseBet, std::move(strat));
22     }
23 };
```

Trong hệ thống, mẫu thiết kế Factory Pattern được triển khai thông qua lớp PlayerFactory. Lớp này chịu trách nhiệm khởi tạo đối tượng Player tương ứng với từng loại chiến lược cược.

Thông qua phương thức `createPlayer`, hệ thống có thể tạo ra các đối tượng người chơi với chiến lược cụ thể (Chung Thủy hoặc Rải Thảm) mà không cần xử lý logic khởi tạo trực tiếp trong các thành phần khác.

Ngoài ra, việc sử dụng `std::unique_ptr` kết hợp với cơ chế chuyển quyền sở hữu (`std::move`) giúp quản lý bộ nhớ hiệu quả, đảm bảo không xảy ra rò rỉ tài nguyên trong quá trình mô phỏng quy mô lớn.

3. Quản lý bộ nhớ hiện đại (Modern Memory Management) - Ứng dụng RAII và Smart Pointers (`std::unique_ptr`)

Trong hệ thống mô phỏng trò Bầu Cua, việc quản lý bộ nhớ đóng vai trò quan trọng do số lượng đối tượng được tạo và hủy rất lớn trong quá trình chạy mô phỏng. Để đảm bảo an toàn và hiệu quả, hệ thống áp dụng nguyên lý RAII kết hợp với Smart Pointer (`std::unique_ptr`) trong C++.

RAII (Resource Acquisition Is Initialization) là một nguyên tắc trong C++ giúp quản lý tài nguyên hiệu quả. Theo nguyên tắc này, tài nguyên như bộ nhớ, tệp hoặc kết nối mạng được cấp phát và giải phóng thông qua các đối tượng. Khi một đối tượng được khởi tạo, nó sẽ nhận tài nguyên cần thiết; khi đối tượng ra khỏi phạm vi, tài nguyên sẽ tự động được giải phóng. RAII giúp tránh rò rỉ bộ nhớ (memory leak) và không cần giải phóng thủ công bằng `delete`.

Trong thực tế, RAII thường được áp dụng thông qua các Smart Pointer như `std::unique_ptr` hoặc `std::shared_ptr`, cho phép tự động quản lý bộ nhớ mà không cần viết mã giải phóng thủ công. Nhờ đó, chương trình trở nên an toàn hơn, giảm thiểu lỗi và dễ dàng bảo trì, đặc biệt trong các hệ thống có quy mô lớn hoặc thực hiện nhiều phép toán lặp lại như mô phỏng Monte Carlo.

Trong project này, `std::unique_ptr` được sử dụng để quản lý vòng đời của các đối tượng quan trọng như: `Player` và `PlayerFactory`. `std::unique_ptr` có đặc điểm như sau:

- Chỉ có một đối tượng sở hữu tài nguyên tại một thời điểm
- Không thể copy, chỉ có thể chuyển quyền sở hữu (move)
- Tự động giải phóng bộ nhớ khi không còn sử dụng

Code minh họa

- `PlayerFactory.h`

```

1  #pragma once
2  #include "Player.h"
3  #include "Strategies.h"
4  #include <memory>
5  #include <string>
6
7  class PlayerFactory {
8  public:
9      enum class Type { CHUNG_THUY, RAI_THAM };
10
11     static std::unique_ptr<Player> createPlayer(Type type, const std::string& name, double balance) {
12         double defaultBaseBet = 100.0;
13         std::unique_ptr<BettingStrategy> strat;
14
15         switch (type) {
16             case Type::CHUNG_THUY:
17                 strat = std::make_unique<ChungThuyStrategy>(); break;
18             case Type::RAI_THAM:
19                 strat = std::make_unique<RaiThamStrategy>(); break;
20         }
21         return std::make_unique<Player>(name, balance, defaultBaseBet, std::move(strat));
22     }
23 };

```

Đoạn code trên sử dụng `std::unique_ptr` để quản lý vòng đời của đối tượng `BettingStrategy`. Việc sử dụng `std::make_unique` giúp cấp phát bộ nhớ an toàn, trong khi `std::move` đảm bảo chuyển quyền sở hữu tài nguyên vào đối tượng `Player`. Khi `Player` bị hủy, tài nguyên sẽ tự động được giải phóng theo nguyên lý RAII.

Code minh họa

- Player.h

```

13     std::unique_ptr<BettingStrategy> strategy;
14
15     public:
16         Player(const std::string& n, double bal, double bBet, std::unique_ptr<BettingStrategy> strat)
17             : name(n), balance(bal), baseBet(bBet), strategy(std::move(strat)) {

```

Đối tượng `Player` sử dụng `std::unique_ptr` để quản lý chiến lược cược (`BettingStrategy`). Việc sử dụng `std::move` trong constructor cho phép chuyển quyền sở hữu tài nguyên vào đối tượng `Player`. Khi đối tượng này bị hủy, tài nguyên sẽ tự động được giải phóng theo nguyên lý RAII, giúp tránh rò rỉ bộ nhớ.

Việc sử dụng RAII và `std::unique_ptr` giúp hệ thống tuân thủ các tiêu chuẩn lập trình hiện đại của C++, đảm bảo hiệu năng và độ ổn định cao. Đây là yếu tố quan trọng giúp hệ thống có thể thực hiện các mô phỏng lớn mà không gặp vấn đề về tài nguyên.

4. Kỹ thuật Lập trình Đa luồng (Multi-threading)

4.1. Khởi tạo luồng động (std::thread::hardware_concurrency)

Trong các bài toán mô phỏng Monte Carlo với số lượng vòng lặp rất lớn (có thể lên đến hàng triệu hoặc hàng trăm triệu lần), việc tối ưu hiệu năng là yếu tố quan trọng. Một trong những giải pháp hiệu quả là áp dụng kỹ thuật lập trình đa luồng (multi-threading) nhằm tận dụng tối đa tài nguyên phần cứng của hệ thống.

Trong C++, hàm `std::thread::hardware_concurrency()` được sử dụng để xác định số lượng luồng tối đa mà CPU có thể hỗ trợ. Dựa trên giá trị này, chương trình có thể phân chia khối lượng công việc thành nhiều phần nhỏ và thực hiện song song trên nhiều luồng khác nhau, từ đó giảm đáng kể thời gian thực thi so với cách xử lý tuần tự.

Code minh họa

- main.cpp

```
23  ✓ int main() {  
24      int totalSimulations = 100000000;  
25  
26      unsigned int numThreads = std::thread::hardware_concurrency();  
27      if (numThreads < 2) numThreads = 4;  
28  
29      unsigned int halfThreads = numThreads / 2;  
30      int simsPerThread = totalSimulations / halfThreads;  
31  }
```

4.2. Đảm bảo tính ngẫu nhiên (Entropy) bằng static thread_local

Trong các hệ thống mô phỏng xác suất, đặc biệt là Monte Carlo, chất lượng của bộ sinh số ngẫu nhiên đóng vai trò rất quan trọng đối với độ chính xác của kết quả. Khi áp dụng đa luồng, nếu nhiều luồng cùng sử dụng chung một bộ sinh số ngẫu nhiên, có thể xảy ra hiện tượng tranh chấp tài nguyên (race condition) hoặc làm giảm tính ngẫu nhiên của dữ liệu.

Để khắc phục vấn đề này, hệ thống sử dụng từ khóa `thread_local` trong C++ để đảm bảo mỗi luồng có một bộ sinh số ngẫu nhiên riêng biệt. Cụ thể, mỗi luồng sẽ sở hữu một đối tượng `std::mt19937` độc lập, được khởi tạo bằng `std::random_device`. Điều này giúp đảm bảo tính ngẫu nhiên (entropy) cao, đồng thời loại bỏ hoàn toàn xung đột giữa các luồng.

Việc sử dụng `thread_local` không chỉ cải thiện độ chính xác của mô phỏng mà còn giúp tăng hiệu năng do không cần sử dụng cơ chế khóa (mutex) để đồng bộ tài nguyên.

Code minh họa

- Strategies.h

```
16         static thread_local std::mt19937 rng(std::random_device{}());
17         std::uniform_int_distribution<int> dist(0, 5);
18
19         Symbol randomSymbol = static_cast<Symbol>(dist(rng));
```

Đoạn mã trên cho thấy mỗi luồng sẽ sử dụng một bộ sinh số ngẫu nhiên riêng để tạo ra kết quả xấp xỉ, đảm bảo tính độc lập và khách quan trong toàn bộ quá trình mô phỏng.

5. Tối ưu hóa hiệu năng I/O (I/O Performance Optimization)

5.1. Giải quyết nút thắt cổ chai ổ cứng (Disk I/O Bottleneck)

Trong các hệ thống mô phỏng Monte Carlo với số lượng vòng lặp rất lớn (hàng triệu đến hàng trăm triệu lần), việc ghi dữ liệu liên tục ra ổ cứng có thể trở thành một nút thắt cổ chai (bottleneck) nghiêm trọng, làm giảm đáng kể hiệu năng của toàn bộ hệ thống.

Nếu chương trình thực hiện ghi file theo từng dòng (line-by-line) sau mỗi vòng lặp, số lần truy cập I/O sẽ cực kỳ lớn. Điều này khiến CPU phải chờ đợi thao tác ghi đĩa hoàn tất, dẫn đến việc không tận dụng hết năng lực xử lý của hệ thống.

Trong bài mô phỏng này, thay vì ghi trực tiếp dữ liệu vào file sau mỗi vòng lặp, hệ thống áp dụng phương pháp ghi theo lô (batch writing). Dữ liệu được tạm thời lưu trữ trong bộ nhớ đệm trước khi ghi xuống ổ cứng theo từng khối lớn. Cách tiếp cận này giúp giảm số lần truy cập I/O, từ đó cải thiện đáng kể tốc độ thực thi chương trình.

5.2. Kỹ thuật Ghi đệm RAM (RAM Buffering) bằng std::ostringstream

Để hiện thực hóa cơ chế ghi theo lô, hệ thống sử dụng lớp std::ostringstream trong C++ như một bộ đệm dữ liệu trong RAM. Thay vì ghi trực tiếp từng dòng vào file, dữ liệu sẽ được ghi vào một buffer tạm thời, sau đó mới được flush (đẩy) xuống file theo từng khối lớn.

Cụ thể, trong quá trình mô phỏng, dữ liệu của mỗi vòng lặp được nối vào buffer. Khi số vòng đạt một ngưỡng nhất định (ví dụ: mỗi 10.000 vòng), toàn bộ nội dung buffer sẽ được ghi xuống file và bộ đệm sẽ được làm rỗng để tiếp tục sử dụng.

Cách làm này mang lại hai lợi ích chính:

- Giảm số lần truy cập ổ cứng → tăng hiệu năng
- Tận dụng tốc độ truy xuất nhanh của RAM

Code minh họa

- Simulator.cpp

```

25     std::ostringstream buffer;
26
27     for (int i = 1; i <= numSimulations; ++i) {
28         std::map<Symbol, int> diceCounts;
29         for (int d = 0; d < 3; ++d) {
30             Symbol result = static_cast<Symbol>(dist(rng));
31             diceCounts[result]++;
32         }
33         bool allPlayersBankrupt = true;
34         buffer << i;
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62         if (i % 10000 == 0) {
63             outFile << buffer.str();
64             buffer.str("");
65             buffer.clear();
66         }
67         if (allPlayersBankrupt || house->isBankrupt()) break;
68     }
69     outFile << buffer.str();
70     outFile.close();
71 }

```

6. Đường ống Dữ liệu Tự động (Automated Data Pipeline) – Tích hợp Python sinh biểu đồ tự động bằng C++ (std::system)

Hệ thống mô phỏng được thiết kế theo mô hình pipeline tự động gồm hai giai đoạn chính:

- **Giai đoạn 1 (C++):** Thực hiện mô phỏng Monte Carlo với hiệu năng cao và xuất dữ liệu kết quả ra file .csv
- **Giai đoạn 2 (Python):** Đọc dữ liệu từ file .csv, xử lý và trực quan hóa bằng biểu đồ

Hai giai đoạn này được liên kết với nhau thông qua cơ chế gọi lệnh hệ thống (std::system), giúp tự động hóa toàn bộ quy trình từ tính toán đến phân tích dữ liệu.

Sau khi quá trình mô phỏng trong C++ hoàn tất, chương trình sẽ tự động gọi các script Python để xử lý dữ liệu và vẽ biểu đồ. Cụ thể, lệnh std::system() được sử dụng để thực thi script Python trực tiếp từ chương trình C++.

Code minh họa

- main.cpp

```
65     std::system("python scripts/bao_cao_nhacai_nho.py");  
66     std::system("python scripts/bao_cao_nhacai_lon.py");  
67
```

Các script Python (sử dụng thư viện pandas, matplotlib, numpy) đảm nhiệm các chức năng:

- Đọc dữ liệu từ file .csv do C++ sinh ra
- Xử lý dữ liệu (làm sạch, chuẩn hóa – ví dụ: forward-fill khi người chơi phá sản sớm)
- Vẽ biểu đồ thể hiện: biến động vốn người chơi, biến động vốn nhà cái, so sánh các chiến thuật cược

Nhờ đó, kết quả mô phỏng được trực quan hóa rõ ràng, hỗ trợ việc phân tích và rút ra kết luận.

Đường ống dữ liệu tự động giúp kết nối hiệu quả giữa tính toán và phân tích. Việc sử dụng `std::system` để tích hợp Python vào C++ giúp hệ thống đạt được cả hiệu năng và khả năng trực quan hóa, từ đó nâng cao giá trị thực tiễn của mô phỏng.

IV. Triển khai mô phỏng và kết quả phân tích

1. Thiết lập các thông số kịch bản mô phỏng

Để tiến hành mô phỏng, hệ thống được thiết lập với các tham số đầu vào nhằm phản ánh hai kịch bản khác nhau của nhà cái: vốn nhỏ và vốn lớn. Các tham số này ảnh hưởng trực tiếp đến kết quả và giúp đánh giá tác động của biến động ngẫu nhiên (variance) cũng như quy luật số lớn (Law of Large Numbers).

1.1. Số vòng mô phỏng

Tổng số vòng mô phỏng được thiết lập ở mức rất lớn nhằm đảm bảo độ tin cậy thống kê.

Ví dụ: Thiết lập chạy 100.000.000 vòng (100 triệu vòng). Số lượng này đủ lớn để quan sát rõ xu hướng dài hạn của các chiến lược.

1.2. Thiết lập đa luồng

Hệ thống sử dụng đa luồng để chia nhỏ khối lượng tính toán:

- Số luồng được lấy tự động từ CPU

- Chia đều cho 2 kịch bản: Nhà cái vốn nhỏ và Nhà cái vốn lớn

Code minh họa

```
23  ✓ int main() {  
24      int totalSimulations = 100000000;  
25  
26      unsigned int numThreads = std::thread::hardware_concurrency();  
27      if (numThreads < 2) numThreads = 4;  
28  
29      unsigned int halfThreads = numThreads / 2;  
30      int simsPerThread = totalSimulations / halfThreads;  
31  }
```

Việc phân chia này giúp tối ưu hiệu năng và rút ngắn thời gian chạy mô phỏng.

1.3. Thiết lập vốn ban đầu

Hai kịch bản được xây dựng nhằm so sánh ảnh hưởng của vốn nhà cái:

- Nhà cái vốn nhỏ: dễ bị phá sản do biến động lớn
- Nhà cái vốn lớn: có khả năng hấp thụ biến động và thể hiện rõ quy luật xác suất

Người chơi được cấp vốn ban đầu cố định và sử dụng các chiến lược cược khác nhau.

1.4. Thiết lập chiến lược cược

Hệ thống sử dụng hai chiến lược chính:

- Chung thủy (Martingale): gấp thếp khi thua
- Rải thảm (Spread Betting): chia vốn cược nhiều cửa

Các người chơi được khởi tạo thông qua Factory Pattern.

```
14      auto house = std::make_unique<House>("NhaCai", houseBalance);  
15      Simulator sim(numSimulations, fileName, std::move(house));  
16  
17      sim.addPlayer(PlayerFactory::createPlayer(PlayerFactory::Type::CHUNG_THUY, "ChungThuy", 800000.0));  
18      sim.addPlayer(PlayerFactory::createPlayer(PlayerFactory::Type::RAI_THAM, "RaiTham", 800000.0));  
19  
20      sim.run();  
21  }
```

1.5. Điều kiện dừng mô phỏng

Mô phỏng sẽ kết thúc khi xảy ra một trong các điều kiện:

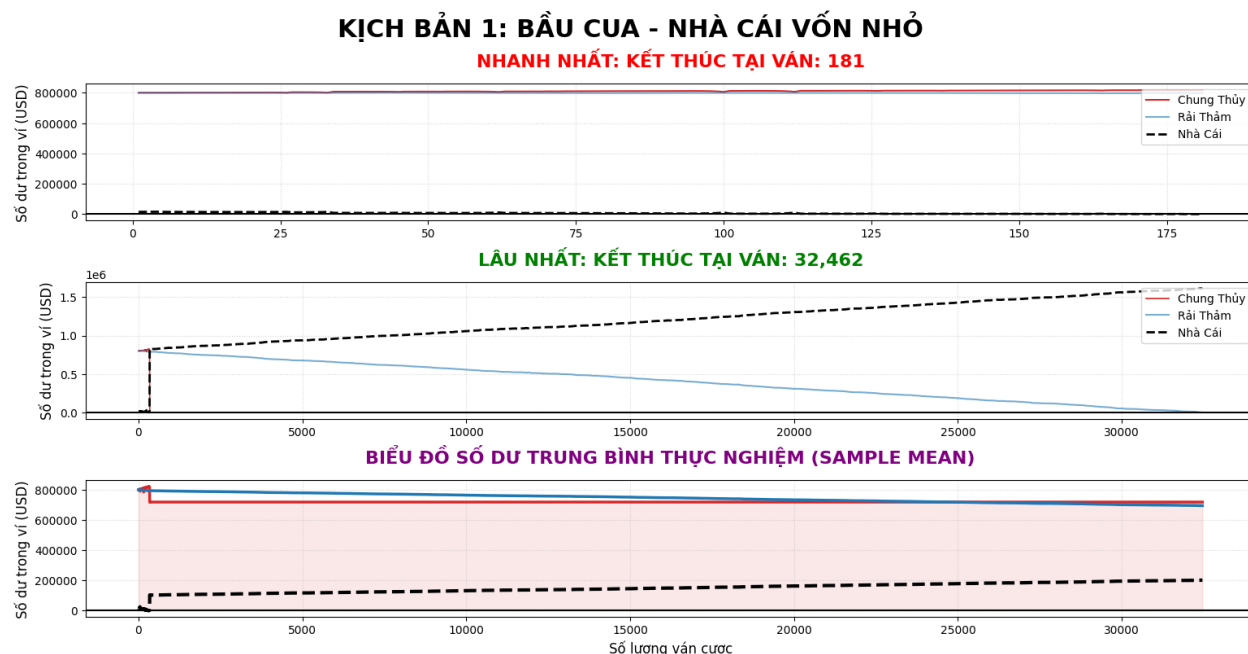
- Người chơi phá sản (balance = 0)

- Nhà cái phá sản

- Hoặc đạt đủ số vòng mô phỏng

Điều này giúp đảm bảo tính thực tế của hệ thống.

2. Kịch bản 1: Nhà cái vốn nhỏ (Mô phỏng Variance ngắn hạn)



2.1. Phân tích kết quả "Người chơi phá sản sớm nhất"

Trong kịch bản này, trường hợp kết thúc nhanh nhất xảy ra tại ván thứ 74, thể hiện rõ trong biểu đồ đầu tiên

Ở giai đoạn đầu, cả hai người chơi đều có mức vốn tương đối ổn định. Tuy nhiên, do áp dụng chiến thuật Chung Thủy (Martingale) – tăng cược theo cấp số nhân sau mỗi lần thua – người chơi nhanh chóng rơi vào tình trạng cược với số tiền lớn vượt quá khả năng chịu đựng của nhà cái.

Khi xuất hiện một chuỗi thắng ngắn, số tiền mà người chơi thắng được tăng đột biến, khiến nhà cái không đủ vốn chi trả. Điều này dẫn đến việc nhà cái bị phá sản chỉ sau một số lượng ván rất nhỏ.

Kết quả này cho thấy trong ngắn hạn, yếu tố ngẫu nhiên (variance) có thể tạo ra những biến động cực lớn, đủ để phá vỡ lợi thế toán học của nhà cái.

2.2. Phân tích kết quả "Người chơi trụ lại lâu nhất"

Trong trường hợp kéo dài lâu nhất, mô phỏng kết thúc tại ván thứ 34,748.

Quan sát biểu đồ thứ hai cho thấy: Người chơi sử dụng chiến thuật Rủi Thảm có xu hướng giảm vốn từ từ theo thời gian. Người chơi Chung Thủy có thể tồn tại lâu hơn nhờ khả năng “gỡ lại” sau chuỗi thua, nhưng vẫn không thể duy trì lâu dài

Trong suốt quá trình này, vốn của nhà cái tăng dần một cách ổn định. Điều này phản ánh đúng bản chất của giá trị kỳ vọng âm (-7.87%), khiến người chơi dần dần mất tiền theo thời gian.

Trường hợp này minh họa rõ ràng:

- Khi số lượng ván đủ lớn, định luật số lớn (Law of Large Numbers) bắt đầu phát huy tác dụng
- Lợi thế của nhà cái trở nên rõ ràng và gần như không thể bị đảo ngược

2.3. Sự sụp đổ của nhà cái dưới áp lực của chiến thuật Martingale

Chiến thuật Martingale (Chung Thủy) là nguyên nhân chính dẫn đến sự sụp đổ của nhà cái trong kịch bản vốn nhỏ. Cơ chế của chiến thuật này là:

- Sau mỗi lần thua, người chơi gấp đôi số tiền cược
- Khi thắng, người chơi thu lại toàn bộ số tiền đã mất trước đó và có lãi

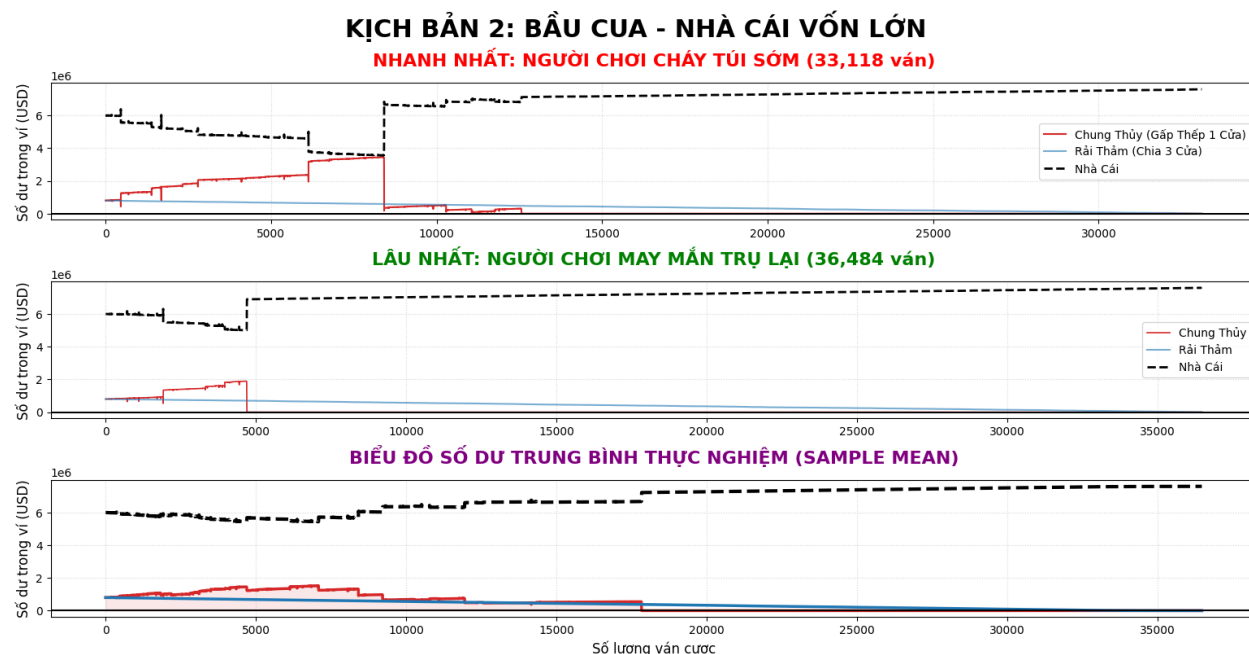
Trong điều kiện nhà cái có vốn nhỏ:

- Một chuỗi thua dài sẽ khiến mức cược tăng lên rất nhanh
- Chỉ cần một lần thắng sau chuỗi thua, người chơi có thể nhận được khoản tiền cực lớn

Biểu đồ cho thấy tại thời điểm gần kết thúc nhanh (ván 74), đường vốn của nhà cái giảm mạnh xuống 0, trong khi người chơi tăng vọt. Đây chính là dấu hiệu của việc nhà cái không đủ khả năng thanh toán (bankrupt).

Tuy nhiên, đây chỉ là hiện tượng trong ngắn hạn. Khi xét trên số lượng ván lớn, chiến thuật Martingale không thể vượt qua lợi thế toán học của nhà cái và vẫn dẫn đến kết cục người chơi phá sản.

3. Kịch bản 2: Nhà cái vốn lớn (Quy luật số lớn)



3.1. Phân tích sự sụt giảm vốn tất yếu của người chơi

Trong kịch bản nhà cái sở hữu nguồn vốn lớn, kết quả mô phỏng cho thấy sự suy giảm vốn của người chơi là một quá trình tất yếu, không phụ thuộc vào chiến lược được áp dụng.

Quan sát hai biểu đồ đầu tiên cho thấy sự khác biệt rõ rệt giữa hai chiến lược. Người chơi áp dụng chiến thuật Chung Thủy (Martingale) có xu hướng gia tăng vốn nhanh trong giai đoạn đầu nhờ các chuỗi thắng ngắn. Tuy nhiên, sự tăng trưởng này không bền vững. Tại khoảng giữa quá trình mô phỏng, đường vốn đạt cực đại và sau đó giảm đột ngột về 0. Nguyên nhân là do sự xuất hiện của một chuỗi thua liên tiếp khiến mức cược tăng theo cấp số nhân, vượt quá khả năng tài chính của người chơi, dẫn đến phá sản ngay lập tức.

Ngược lại, người chơi sử dụng chiến thuật Rải Thảm thể hiện một quỹ đạo ổn định hơn. Vốn giảm dần theo thời gian với biên độ dao động nhỏ, không xuất hiện các cú sụt giảm đột ngột. Tuy nhiên, xu hướng chung vẫn là giảm liên tục và cuối cùng tiến về 0 sau một số lượng ván lớn.

Trong toàn bộ quá trình, vốn của nhà cái không chịu ảnh hưởng đáng kể từ các biến động ngắn hạn. Đường biểu diễn cho thấy xu hướng tăng trưởng ổn định, đặc biệt rõ rệt sau khi người chơi phá sản.

Từ đó có thể rút ra nhận định:

- Chiến thuật Martingale làm gia tăng tốc độ phá sản do rủi ro tích lũy cao
- Chiến thuật Rải Thảm giúp kéo dài thời gian tồn tại nhưng không thay đổi kết cục

Như vậy, khi nhà cái có đủ vốn để hấp thụ biến động, mọi chiến lược của người chơi đều dẫn đến sự suy giảm vốn và kết thúc bằng phá sản.

3.2. Biến động số dư trung bình thực nghiệm

Từ phân tích ở phần “II. CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH TOÁN HỌC”, ta xác định được kỳ vọng EV của trò Bầu Cua xấp xỉ -0.0787 . Giá trị này cho thấy, về trung bình, người chơi mất khoảng 7.87% giá trị cược sau mỗi ván. Đây là một đặc trưng cố hữu của trò chơi và không phụ thuộc vào cách đặt cược.

Phân tích biểu đồ thứ ba cho thấy xu hướng hội tụ rõ rệt của các đường vốn theo thời gian. Mặc dù trong giai đoạn đầu tồn tại những dao động mạnh do yếu tố ngẫu nhiên, nhưng khi số lượng ván tăng lên, các dao động này dần bị triệt tiêu. Cụ thể như sau:

- Đường vốn của cả hai chiến lược đều có xu hướng giảm dần về 0
- Chiến thuật Chung Thủy tạo ra các biến động lớn và dẫn đến phá sản đột ngột
- Chiến thuật Rải Thảm cho đường cong mượt hơn nhưng vẫn giảm liên tục
- Đường vốn của nhà cái tăng trưởng ổn định theo thời gian, phản ánh trực tiếp lợi thế kỳ vọng dương của phía nhà cái.

Hiện tượng này được giải thích bởi định luật số lớn (Law of Large Numbers). Theo đó, khi số lần thử đủ lớn, giá trị trung bình thực nghiệm sẽ tiến gần đến giá trị kỳ vọng lý thuyết. Do kỳ vọng của trò chơi là âm đối với người chơi, nên trong dài hạn, đường vốn sẽ tất yếu có xu hướng giảm về 0.

Đáng chú ý, sự khác biệt giữa các chiến lược chỉ thể hiện ở mức độ biến động trong ngắn hạn. Chiến thuật Chung Thủy tạo ra các dao động lớn và dẫn đến phá sản đột ngột, trong khi chiến thuật Rải Thảm làm giảm biên độ dao động và kéo dài thời gian tồn tại. Tuy nhiên, cả hai đều hội tụ về cùng một xu hướng dài hạn do chịu sự chi phối của cùng một giá trị kỳ vọng âm.

4. Đánh giá chung về mức độ rủi ro của các chiến thuật

Dựa vào kết quả mô phỏng hai kịch bản Nhà cái vốn lớn và Nhà cái vốn nhỏ, có thể nhận thấy rằng mức độ rủi ro của các chiến thuật đặt cược trong trò Bầu Cua không chỉ phụ thuộc vào cách phân bổ vốn mà còn chịu chi phối mạnh mẽ bởi bản chất xác suất của trò chơi. Mặc dù mỗi chiến thuật thể hiện những đặc điểm khác nhau về biến động và thời gian tồn tại, nhưng tất cả đều không thể vượt qua giá trị kỳ vọng âm vốn có.

Chiến thuật Chung Thủy (Martingale) được đánh giá là chiến thuật có mức độ rủi ro cao nhất. Cơ chế tăng cược theo cấp số nhân sau mỗi lần thua giúp người chơi có khả năng thu hồi vốn nhanh chóng khi gặp một lần thắng. Tuy nhiên, chính đặc điểm này lại khiến chiến thuật trở nên cực kỳ nguy hiểm. Khi xuất hiện một chuỗi thua dài – điều không thể tránh khỏi trong một quá trình ngẫu nhiên – mức cược sẽ tăng rất nhanh và vượt quá khả năng tài chính của người chơi. Kết quả là người chơi rơi vào trạng thái phá sản đột ngột.

Ngược lại, chiến thuật Rải Thảm (Spread Betting) có mức độ rủi ro thấp hơn về mặt biến động. Việc phân bổ vốn vào nhiều cửa cùng lúc giúp giảm thiểu tác động của các kết quả bất lợi trong từng ván, từ đó làm cho đường vốn ổn định hơn và ít xuất hiện các cú sốc lớn. Tuy nhiên, sự ổn định này chỉ mang tính tương đối trong ngắn hạn. Do vẫn chịu ảnh hưởng của cùng một giá trị kỳ vọng âm, người chơi sử dụng chiến thuật này sẽ dần dần mất vốn theo thời gian và cuối cùng vẫn đi đến trạng thái phá sản. Nói cách khác, chiến thuật này không làm thay đổi kết quả cuối cùng mà chỉ kéo dài quá trình thua lỗ.

Ngoài ra, kết quả mô phỏng trong kịch bản nhà cái vốn lớn còn cho thấy một điểm quan trọng: khi nhà cái có đủ nguồn lực tài chính để hấp thụ mọi biến động ngắn hạn, rủi ro phá sản gần như chỉ tồn tại ở phía người chơi. Điều này làm nổi bật bản chất bất cân xứng của trò chơi, trong đó lợi thế dài hạn luôn nghiêng về phía nhà cái.

Có thể khẳng định rằng không tồn tại chiến thuật đặt cược nào có thể loại bỏ hoặc đảo ngược rủi ro trong trò chơi Bầu Cua. Mọi chiến thuật chỉ có thể điều chỉnh mức độ biến động và thời gian tồn tại, nhưng không thể thay đổi kết quả cuối cùng. Đây là hệ quả tất yếu của một hệ thống có giá trị kỳ vọng âm đối với người chơi.

5. Đề xuất chiến lược tối ưu

5.1. Góc nhìn Người chơi: Tham gia như thế nào là tối ưu nhất?

Dựa trên các phân tích về giá trị kỳ vọng và kết quả mô phỏng, có thể khẳng định rằng không tồn tại chiến lược đặt cược nào giúp người chơi đạt lợi nhuận ổn định trong dài hạn. Nguyên nhân cốt lõi là do trò chơi mang giá trị kỳ vọng âm ($EV < 0$), khiến người chơi về mặt toán học luôn ở vị thế bất lợi. Do đó, "tối ưu nhất" không nằm ở việc tìm ra công thức chiến thắng tuyệt đối, mà là chiến lược quản trị rủi ro:

Chiến lược tối ưu như sau:

- Tuyệt đối từ bỏ ảo tưởng Gấp thếp (Chung Thủy): Mặc dù Kịch bản 1 chỉ ra rằng người chơi có thể làm phá sản Nhà cái nếu có số vốn áp đảo nhưng trong thực tế (Kịch bản 2), Nhà cái luôn sở hữu nguồn vốn khổng lồ. Việc nhân đôi tiền cược sau mỗi ván thua là một cạm bẫy cực kỳ nguy hiểm. Nó ép người chơi phải đánh cược toàn bộ tài sản chỉ để đổi lấy mức lợi nhuận bằng đúng ván cược đầu tiên. Thực nghiệm cho thấy những luồng chơi áp dụng chiến thuật này luôn là những người "cháy túi" nhanh nhất.
- Sử dụng "Rải Thảm" làm khiên phòng thủ: Nếu mục đích tham gia là để giải trí và trải nghiệm trò chơi lâu nhất có thể, hãy chọn chiến thuật Rải Thảm. Việc chia nhỏ số tiền cược và rải đều vào 3 cửa ngẫu nhiên giúp giảm thiểu phương sai (độ lệch chuẩn) của sự may rủi. Dữ liệu chứng minh chiến thuật này giúp người chơi có đồ thị tài sản dao động ổn định hơn và trụ lại trên bàn cược lâu hơn gấp nhiều lần so với đánh tất tay vào một cửa.
- Kỷ luật "Đánh nhanh rút gọn" (Hit and Run): Kẻ thù lớn nhất của người chơi là biến số thời gian (số lượng ván cược). Trò chơi càng kéo dài, Lợi thế nhà cái (House Edge) càng phát huy tác dụng. Cách duy nhất để mang tiền về là tận dụng những chuỗi thắng ngẫu nhiên ở giai đoạn ngắn hạn. Người chơi bắt buộc phải đặt ra mức Chốt lời (Take-profit) và Cắt lỗ (Stop-loss) nghiêm ngặt trước khi chơi, và lập tức rời bàn khi chạm một trong hai ngưỡng này, tuyệt đối không bị cuốn vào vòng lặp của trò chơi.

5.2. Góc nhìn Chủ (Nhà cái): Quản lý trò chơi như thế nào để tối ưu lợi nhuận?

Nhà Cái đã có sẵn lợi thế về mặt toán học, nhưng mô phỏng cho thấy họ vẫn có thể phá sản nếu quản lý dòng tiền kém. Để tối ưu hóa và đảm bảo lợi nhuận tuyệt đối, Nhà Cái cần áp dụng các chiến lược sau:

- Chuẩn bị nguồn Vốn dự phòng khổng lồ: Kịch bản 2 khả năng: Vốn càng dày, Nhà Cái càng "bắt từ". Nguồn vốn lớn đóng vai trò là "bộ đệm" hấp thụ mọi đợt sóng thắng đột biến hoặc các pha xả tiền gấp thếp của người chơi, kéo ván cược về đúng quỹ đạo đường cong EV.
- Thiết lập Giới hạn cược trần (Max Bet / Table Limit): Đây là "vũ khí" chí mạng để bẻ gãy chiến thuật Chung Thủy (Gấp thếp). Khi áp đặt mức cược tối đa cho mỗi cửa, người chơi sẽ không thể tiếp tục nhân đôi tiền cược khi đang trong chuỗi thua, khiến họ buộc phải chấp nhận mất lỗ mà không có cơ hội gỡ lại bằng vốn.
- Tối đa hóa số lượng ván cược (Volume): Lợi nhuận của Nhà Cái không đến từ một vài ván thắng lớn, mà đến từ tỷ lệ lợi thế nhỏ lẻ (House Edge) tích lũy qua hàng triệu giao dịch. Chiến lược quản lý tối ưu là tổ chức trò chơi nhanh gọn, liên tục và giữ chân người chơi trên bàn càng lâu càng tốt để "Quy luật số lớn" phát huy sức mạnh

V. Kết luận và hướng phát triển

1. Kết quả đạt được về mặt Kỹ thuật Phần mềm

Dự án đã xây dựng thành công một hệ thống mô phỏng lai (Hybrid Architecture) kết hợp sức mạnh tính toán của C++ và khả năng trực quan hóa của Python. Về mặt kỹ thuật, hệ thống đạt được những điểm sáng sau:

- Kiến trúc lai C++/Python hiệu năng cao: Đồ án đã thiết lập thành công một quy trình (pipeline) xử lý dữ liệu chuẩn xác, kết hợp tốc độ mô phỏng cực nhanh của C++ (đóng vai trò như một động cơ lõi) và khả năng xử lý mảng dữ liệu, vẽ biểu đồ trực quan của Python.
- Ứng dụng Mẫu thiết kế (Design Patterns) chuẩn mực: Mã nguồn C++ tuân thủ chặt chẽ lập trình hướng đối tượng. Mẫu Strategy được dùng để tách biệt logic đặt cược khỏi thực thể Người chơi, và mẫu Factory giúp tự động hóa quá trình khởi tạo đối tượng. Điều này giúp hệ thống dễ dàng bảo trì và mở rộng thêm các chiến thuật mới mà không phá vỡ cấu trúc cũ.
- Tối ưu hóa Đa luồng (Lock-free Multithreading): Điểm sáng kỹ thuật lớn nhất là việc thực thi 100.000.000 lượt mô phỏng song song mà không vướng phải điểm nghẽn đồng bộ hóa. Hệ thống tự động chia tải dựa trên sức mạnh phần cứng `std::thread::hardware_concurrency()` và cách ly hoàn toàn trạng thái ngẫu nhiên bằng cơ chế `static thread_local std::mt19937`, bảo vệ tính độc lập của thuật toán Monte Carlo.

-Quản lý bộ nhớ an toàn và Tối ưu I/O: Khắc phục được nút thắt cổ chai ở đĩa cứng bằng kỹ thuật bộ đệm RAM (chỉ ghi file sau mỗi 10.000 ván). Về mặt hướng đối tượng, hệ thống vận dụng nguyên tắc RAII thông qua con trỏ thông minh `std::unique_ptr`, đảm bảo vòng đời của các thực thể chiến thuật (Strategy) và người chơi (Player) được quản lý chặt chẽ, loại trừ nguy cơ rò rỉ RAM (memory leak) trong thời gian dài.

2. Kết quả đạt được về mặt Thống kê (Bản chất của sòng bạc)

Hệ thống không chỉ hoạt động trơn tru mà còn dùng dữ liệu lớn (Big Data) để chứng minh trực quan các định lý toán học trong ngành công nghiệp cờ bạc:

- Sức mạnh của Luật Số Lớn (Law of Large Numbers): Biểu đồ "Đường cong kỳ vọng trung bình (EV)" ở cả hai kịch bản đều chứng minh: Trong dài hạn, bất kể người chơi dùng chiến thuật gì, dòng tiền trung bình của họ luôn hướng về 0, trong khi lợi nhuận của Nhà cái tăng lên ổn định.
- Phơi bày điểm yếu của chiến thuật Gấp thếp (Martingale): Người chơi Chung Thủy (đánh 1 cửa, nhân đôi tiền khi thua) được ghi nhận là đối tượng cháy túi nhanh nhất trên bàn cược. Tốc độ tăng trưởng cấp số nhân của tiền cược nhanh chóng vắt kiệt số vốn hữu hạn của người chơi.
- Chiến thuật Phân tán rủi ro chỉ làm chậm quá trình: Mặc dù chiến thuật "Rải Thảm" (chia cược vào 3 cửa bằng tối đa 1/3 vốn) giúp nén biến động (variance) và kéo dài sự sinh tồn của người chơi, nó vẫn không chiến thắng được giá trị kỳ vọng (EV) âm. Quy luật Số lớn từ từ rút cạn dòng tiền của người chơi một cách êm ái.
- Sức mạnh của chiều sâu thanh khoản: Đồ án làm nổi bật sự khác biệt giữa hai rào cản tài chính. "Nhà cái nhỏ" (15.000 USD) dễ dàng bị đập vỡ nợ bởi phương sai ngắn hạn của người chơi liều lĩnh. Trong khi đó, "Nhà cái lớn" (6.000.000 USD) như một hố đen, đủ sức hấp thụ toàn bộ các cú sốc xác suất, biến rủi ro thành lợi nhuận định lượng ổn định. Phương pháp nội suy chuỗi thời gian (Forward-filling) được sử dụng đã loại bỏ thiên lệch sinh tồn, phản ánh quỹ đạo đi xuống chân thực của hệ thống.

3. Hạn chế còn tồn tại của hệ thống

Mặc dù hoàn thiện về cấu trúc cốt lõi, dự án vẫn còn một số điểm giới hạn cần khắc phục:

- Thiếu quy tắc "Giới hạn cược trần" (Max Bet / Table Limit): Trong mã nguồn hiện tại, người chơi được phép đặt cược số tiền lên tới giới hạn bằng toàn bộ số dư trong ví của họ (amount =

`std::min(amount, balance);`). Điều này tạo ra một lỗ hổng phi thực tế so với các sòng bạc đời thực (nơi luôn có Max Bet để chủ động cắt đứt chuỗi Gấp thếp của người chơi).

- Chưa tối ưu thuật toán cấp phát bộ nhớ: Trong lớp `RaiThamStrategy`, mỗi khi tính toán tiền cược, hệ thống lại phải khởi tạo một `std::vector` mới chứa 6 mặt xúc xắc và thực hiện hàm `std::shuffle`. Khi chạy hàng trăm triệu ván cược, việc cấp phát và thu hồi bộ nhớ liên tục này gây lãng phí chu kỳ CPU và làm chậm tốc độ mô phỏng tổng thể.

- Nút thắt về lưu trữ dữ liệu thô: Việc xuất hàng loạt log giao dịch ra các định dạng văn bản .csv qua ổ đĩa tiêu tốn lượng lớn băng thông và dung lượng. Khối lượng tệp khổng lồ này khiến kịch bản Python tốn khá nhiều thời gian để gom nhóm dữ liệu (glob) và nội suy mảng.

- Mô hình hành vi còn tĩnh: Các quy tắc của "Chung Thủy" hay "Rải Thảm" vẫn mang tính máy móc thuần túy theo công thức. Đồ án chưa thể hiện được các chỉ số tâm lý học hành vi cực kỳ quan trọng trong đầu tư hay cờ bạc như lòng tham (fomo), sự hoảng loạn hay sự bỏ cuộc sớm (early exit).

- Hạn chế trong biểu diễn và diễn giải giá trị kỳ vọng (EV): Trong phiên bản ban đầu, nhóm sử dụng thuật ngữ “biểu đồ đường cong kỳ vọng (EV)” để mô tả đường trung bình vốn của người chơi theo thời gian. Tuy nhiên, cách gọi này chưa hoàn toàn chính xác về mặt lý thuyết xác suất. Cụ thể, giá trị kỳ vọng lý thuyết của trò chơi là một đại lượng ổn định và thường được biểu diễn dưới dạng xu hướng tuyến tính, trong khi biểu đồ của nhóm được xây dựng dựa trên trung bình của số lượng mẫu mô phỏng hữu hạn (khoảng 8 mẫu). Do kích thước mẫu nhỏ và chịu ảnh hưởng lớn từ biến động ngẫu nhiên (đặc biệt từ chiến thuật Chung Thủy – Martingale), đường biểu diễn xuất hiện hiện tượng dao động và đứt đoạn. Điều này có thể gây hiểu nhầm khi đánh đồng với kỳ vọng toán học. Vì vậy, nhóm đã điều chỉnh lại thuật ngữ thành “biểu đồ số dư trung bình thực nghiệm”, phản ánh đúng bản chất dữ liệu thu được từ mô phỏng. Hạn chế này cho thấy hệ thống hiện tại chưa đủ dữ liệu để thể hiện rõ sự hội tụ theo định luật số lớn, đồng thời chưa tách biệt hoàn toàn giữa kỳ vọng lý thuyết và giá trị thực nghiệm.

4. Hướng phát triển trong tương lai

Để biến đồ án này thành một hệ thống mô phỏng chuẩn nghiệp vụ phân tích rủi ro (Risk Analysis), các bước phát triển tiếp theo bao gồm:

- Bổ sung Business Logic thực tế: Tích hợp biến `tableMinBet` và `tableMaxBet` vào lớp `House` hoặc `Simulator` để giới hạn khoản tiền đặt cược, mô phỏng chính xác hơn môi trường sòng bạc thực tế.

- Tối ưu hóa Micro-performance: Cải tiến lại lớp RaiThamStrategy bằng cách sử dụng các mảng tĩnh (Static Arrays) hoặc các thuật toán lấy mẫu ngẫu nhiên không cần cấp phát lại bộ nhớ để tiết kiệm tài nguyên tính toán.
- Đa dạng hóa hành vi con người (Dynamic Strategies): Nâng cấp "Bộ não" của Player để họ có thể thay đổi chiến thuật linh hoạt (Ví dụ: Đang thắng thì chuyển sang Rủi thăm bảo toàn vốn, đang thua thì chuyển sang Gấp thếp, hoặc thiết lập điểm chốt lời/cắt lỗ tự động rời bàn).
- Tối ưu hóa Luồng Dữ liệu (Data Pipeline): Thay vì sử dụng file CSV cồng kềnh, hệ thống nên chuyển đổi sang các định dạng lưu trữ nhị phân hiện đại như HDF5 hoặc Apache Parquet. Bổ sung kiến trúc truyền phát thời gian thực (ví dụ như Apache Kafka kết hợp WebSockets) để xây dựng bảng điều khiển báo cáo trực tiếp thay vì chạy script thông dịch sau cùng.
- Mở rộng Điện toán Phân tán (Distributed System): Nâng cấp engine mô phỏng vượt khỏi giới hạn của một máy tính đơn lẻ. Có thể triển khai trên kiến trúc giao tiếp tin nhắn (MPI) hoặc các cụm điện toán đám mây (Cloud Computing) để phân chia hàng tỷ lượt chạy trên nhiều Server Node khác nhau.
- Đưa Trí tuệ Nhân tạo (Machine Learning) vào mô hình: Thay thế các giao diện BettingStrategy cứng nhắc bằng các mạng nơ-ron Học tăng cường (Reinforcement Learning Agents). Máy học có thể liên tục thử nghiệm các chính sách quản lý vốn phức tạp nhằm tìm ra kẻ hờ xác suất, biến công cụ này thành một bộ giả lập rủi ro thực thụ.
- Tổng quát hóa Engine Mô phỏng: Trừu tượng hóa hoàn toàn không gian xác suất để không chỉ chạy giới hạn ở luật Bầu Cua (216 trạng thái). Hướng tới việc cấu hình hóa tham số để hệ thống có thể giả lập bất kỳ rủi ro nào như tỷ giá chứng khoán phái sinh, quản trị rủi ro thanh khoản quỹ đầu tư, hay phân tích sụp đổ dây chuyền trong tài chính.
- Nâng cấp Giao diện (Dashboard): Thay thế việc xuất file đồ thị ảnh tĩnh bằng Python bằng việc xây dựng một giao diện Web/GUI tương tác (ví dụ sử dụng Dash hoặc Streamlit), cho phép người dùng nhập tham số mô phỏng (số vốn, tỷ lệ cược) và xem biểu đồ chạy theo thời gian thực (Real-time).
- Cải thiện độ chính xác thống kê của biểu đồ: Tăng số lượng mẫu mô phỏng độc lập (thay vì chỉ sử dụng một số lượng nhỏ mẫu), đồng thời bổ sung đường biểu diễn kỳ vọng lý thuyết để so sánh với trung bình thực nghiệm. Ngoài ra, có thể áp dụng các kỹ thuật làm mượt dữ liệu (moving

average) hoặc hiển thị khoảng tin cậy (confidence interval) nhằm thể hiện rõ hơn xu hướng hội tụ và giảm nhiễu do biến động ngẫu nhiên.

VI. Tổng kết

1. Tổng kết

Đồ án đã xây dựng thành công một hệ thống mô phỏng hiệu năng cao nhằm phân tích bài toán Gambler's Ruin trong bối cảnh trò chơi Bầu Cua, kết hợp giữa lập trình hướng đối tượng, kỹ thuật đa luồng trong C++ và phân tích dữ liệu bằng Python.

Về mặt kỹ thuật, hệ thống thể hiện rõ năng lực thiết kế phần mềm hiện đại thông qua việc áp dụng các nguyên lý Lập trình hướng đối tượng (OOP), sử dụng các mẫu thiết kế như Strategy Pattern và Factory Pattern, cùng với cơ chế quản lý bộ nhớ an toàn bằng RAII và `std::unique_ptr`. Bên cạnh đó, việc triển khai đa luồng với `std::thread` và `thread_local` giúp tối ưu hóa hiệu năng, cho phép mô phỏng hàng trăm triệu ván chơi trong thời gian ngắn.

Về mặt toán học – thống kê, kết quả mô phỏng đã xác nhận các kết luận lý thuyết:

- Giá trị kỳ vọng (EV) của trò chơi là âm ($\sim -7.87\%$), dẫn đến xu hướng thua lỗ tất yếu của người chơi.
- Chiến thuật Martingale tạo ra biến động lớn và dẫn đến phá sản nhanh.
- Chiến thuật Rủi thảm giúp ổn định hơn nhưng không thay đổi kết cục dài hạn.

2. Tuyên bố miễn trừ trách nhiệm

Đồ án này được thực hiện hoàn toàn với mục đích học thuật và nghiên cứu xác suất – thống kê, nhằm minh họa các khái niệm như giá trị kỳ vọng, biến động (variance) và định luật số lớn trong các hệ thống ngẫu nhiên.

Mô hình và kết quả trong đề tài không nhằm mục đích khuyến khích hoặc cổ vũ hành vi cờ bạc dưới bất kỳ hình thức nào. Thay vào đó, hệ thống được xây dựng để cung cấp một góc nhìn khoa học, giúp người học hiểu rõ hơn về rủi ro tài chính và bản chất bất lợi của các trò chơi may rủi.

3. Hướng dẫn cài đặt và sử dụng mã nguồn

- Yêu cầu hệ thống

- Trình biên dịch C++ hỗ trợ C++17 (khuyến nghị: Visual Studio 2022)
- Python phiên bản 3.x
- Hệ điều hành: Windows (khuyến nghị), hoặc Linux/macOS có cấu hình tương đương

- Cài đặt thư viện Python

- Hệ thống sử dụng Python để xử lý dữ liệu và vẽ biểu đồ. Cài đặt các thư viện cần thiết bằng lệnh: `pip install pandas matplotlib numpy`

- Chuẩn bị môi trường

- Tạo thư mục data/ trong thư mục gốc của project để lưu dữ liệu mô phỏng
- Đảm bảo biến môi trường đã nhận diện lệnh python (hoặc python3 tùy hệ điều hành)

- Cách chạy chương trình

- Mở file project (.sln) bằng Visual Studio 2022
- Build và chạy chương trình
- Hệ thống sẽ thực hiện:
 - Mô phỏng Monte Carlo bằng C++
 - Xuất dữ liệu ra file .csv
 - Tự động gọi script Python để trực quan hóa kết quả

- Trường hợp Python không tự động chạy sau khi quá trình chạy C++ kết thúc:

- Hệ thống C++ sử dụng lệnh `std::system("python ...")` để kích hoạt các script Python vẽ biểu đồ. Trong một số trường hợp, biểu đồ có thể không hiển thị sau khi chương trình C++ kết thúc.
- Nguyên nhân thường do biến môi trường của hệ điều hành chưa nhận diện đúng lệnh python. Để khắc phục:
 - Cần kiểm tra và đảm bảo biến môi trường (Environment Variable) đã được thiết lập đúng.
 - Đối với macOS/Linux hoặc các phiên bản Python mới (sử dụng python3), người dùng có thể chạy thủ công các script bằng lệnh:


```
python3 scripts/bao_cao_nhacai_nho.py
python3 scripts/bao_cao_nhacai_lon.py
```


VII. Tài liệu tham khảo

Tài liệu Tiếng Anh

- [1] W. Kenton, "Investopedia," 24 3 2026. [Online]. Available:
<https://www.investopedia.com/terms/m/montecarlosimulation.asp>. [Accessed 4 5 2026].

Tài liệu Tiếng Việt

- [2] B. Learn, "Bybit Learn," 22 11 2023. [Online]. Available:
<https://learn.bybit.com/vi/trading/what-is-martingale-strategy>. [Accessed 4 5 2026].